

Mise en place d'une infrastructure DevOps avec CI/CD et Kubernetes pour une application de gestion des absences

Présentée par :

Ouarrak Layla

Sanae Outighli

Encadré par :

Pr. F. BENABBOU

Département :

Informatique et mathématique

Filière :

Master d'excellence IA

Année Universitaire : 2026 - 2027

Table des matières

Liste de figures	2
1. Introduction	3
1.1 Présentation générale du projet	3
1.2 Contexte et importance du DevOps	3
1.3 Objectifs du projet	3
2. Étude préalable	4
2.1 Analyse des besoins	4
2.2 Choix des outils et technologies	4
3. Conception	6
3.1 Architecture du système	6
3.2 Description des composantes	7
4. Implémentation	8
4.1 Création de l'application	8
4.2 Conteneurisation avec Docker	11
4.3 Infrastructure as Code avec Terraform	12
4.4 Configuration automatique avec Ansible	14
4.5 Déploiement avec Kubernetes (K3s)	17
4.6 Pipeline CI/CD avec GitLab	18
5. Résultats et validation	20
5.1 Tests réalisés	20
5.2 Résultats obtenus	20
6. Conclusion et perspectives	21
6.1 Bilan	21
6.2 Limites	21
6.3 Améliorations futures	21

Liste de figures

Image 1 : Architecture globale de l'infrastructure DevOps	6
Image 2 : Structure du projet GestiAbsence	8
Image 3 : Interface principale de l'application GestiAbsence	9
Image 4 : Interface d'ajouter une absence	10
Image 5 : Page de statistiques analytiques	10
Image 6 : Dockerfile de l'application	11
Image 7 : Construction et push de l'image Docker	12
Image 8 : Fichiers Terraform (main.tf, variables.tf, outputs.tf)	13
Image 9 : Résultat de terraform apply — 2 VMs créées	13
Image 10 : Fichiers Ansible (inventory.ini , playbook.yml)	15
Image 11 : Résultat ansible ping — SUCCESS sur les 2 nœuds	16
Image 12 : Fichier deployment.yaml Kubernetes	17
Image 13 : État des pods et des nœuds K3s	17
Image 14 : Fichier .gitlab-ci.yml complet	18
Image 15 : Variables CI/CD dans GitLab	19
Image 16 : Pipeline GitLab réussi — 4 stages verts	19
Image 17 : Application accessible sur http://13.48.46.222:30080	20

1. Introduction

1.1 Présentation générale du projet

Dans le cadre du module DevOps du Master DSBD & IA, ce projet a pour objectif principal de mettre en œuvre une infrastructure complète permettant le déploiement automatisé d'une application dans un environnement proche des conditions réelles de production.

L'application choisie comme support est GestiAbsence, une application web de gestion des absences développée avec Flask en Python. Elle offre une interface web permettant de centraliser et gérer les absences des étudiants.

Le projet ne se limite pas au développement de l'application, mais vise surtout à appliquer les principes DevOps en intégrant des outils modernes : Docker, Kubernetes (K3s), Terraform, Ansible et GitLab CI/CD.

1.2 Contexte et importance du DevOps

Dans les environnements informatiques actuels, les entreprises recherchent des méthodes de déploiement plus rapides, plus fiables et mieux automatisées. L'approche DevOps répond à ce besoin en favorisant l'automatisation de toutes les étapes du cycle de vie d'une application.

Grâce à la conteneurisation avec Docker, l'orchestration avec Kubernetes, et les pipelines CI/CD avec GitLab, il devient possible de réduire les erreurs humaines, d'améliorer la reproductibilité et d'assurer un déploiement continu et fiable.

1.3 Objectifs du projet

Les principaux objectifs du projet sont :

- Développer une application web simple (GestiAbsence) comme support d'expérimentation
- Conteneuriser l'application avec Docker
- Automatiser la création de l'infrastructure cloud avec Terraform
- Configurer les machines automatiquement avec Ansible
- Déployer l'application sur un cluster Kubernetes (K3s)
- Mettre en place une chaîne CI/CD complète avec GitLab
- Comprendre l'intégration des différents outils DevOps dans une architecture complète

2. Étude préalable

2.1 Analyse des besoins

Avant de mettre en place l'infrastructure DevOps, il est nécessaire d'identifier les besoins fonctionnels et techniques du projet.

✧ **Besoins fonctionnels**

- Ajouter, modifier et supprimer des absences
- Consulter la liste des absences
- Afficher des statistiques analytiques
- Authentification administrateur
- Export des données en Excel

✧ **Besoins techniques**

- Exécuter l'application dans un environnement isolé et reproductible
- Automatiser le build, les tests et le déploiement
- Assurer la haute disponibilité avec plusieurs replicas
- Permettre une gestion simple de l'infrastructure cloud

2.2 Choix des outils et technologies

Outil	Rôle	Justification
Python / Flask	Développement de l'application web	Framework léger, rapide à développer
Docker	Conteneurisation de l'application	Portabilité et isolation de l'environnement
Kubernetes (K3s)	Orchestration des conteneurs	Gestion automatique, haute disponibilité
Terraform	Infrastructure as Code	Création automatique des VMs AWS
Ansible	Configuration automatique	Installation sans intervention manuelle

Outil	Rôle	Justification
GitLab CI/CD	Pipeline automatisé	Build, test, push et deploy automatiques
Docker Hub	Registre d'images	Stockage et distribution des images Docker
AWS EC2	Hébergement cloud	Machines virtuelles gratuites (t3.micro)

Table 1 : Tableau des choix des outils et technologies

3. Conception

3.1 Architecture du système

L'architecture du projet repose sur une infrastructure DevOps complète organisée en plusieurs couches distinctes. L'utilisateur interagit avec l'application via un navigateur web. Le flux de déploiement automatisé part du développeur jusqu'à l'application en production.

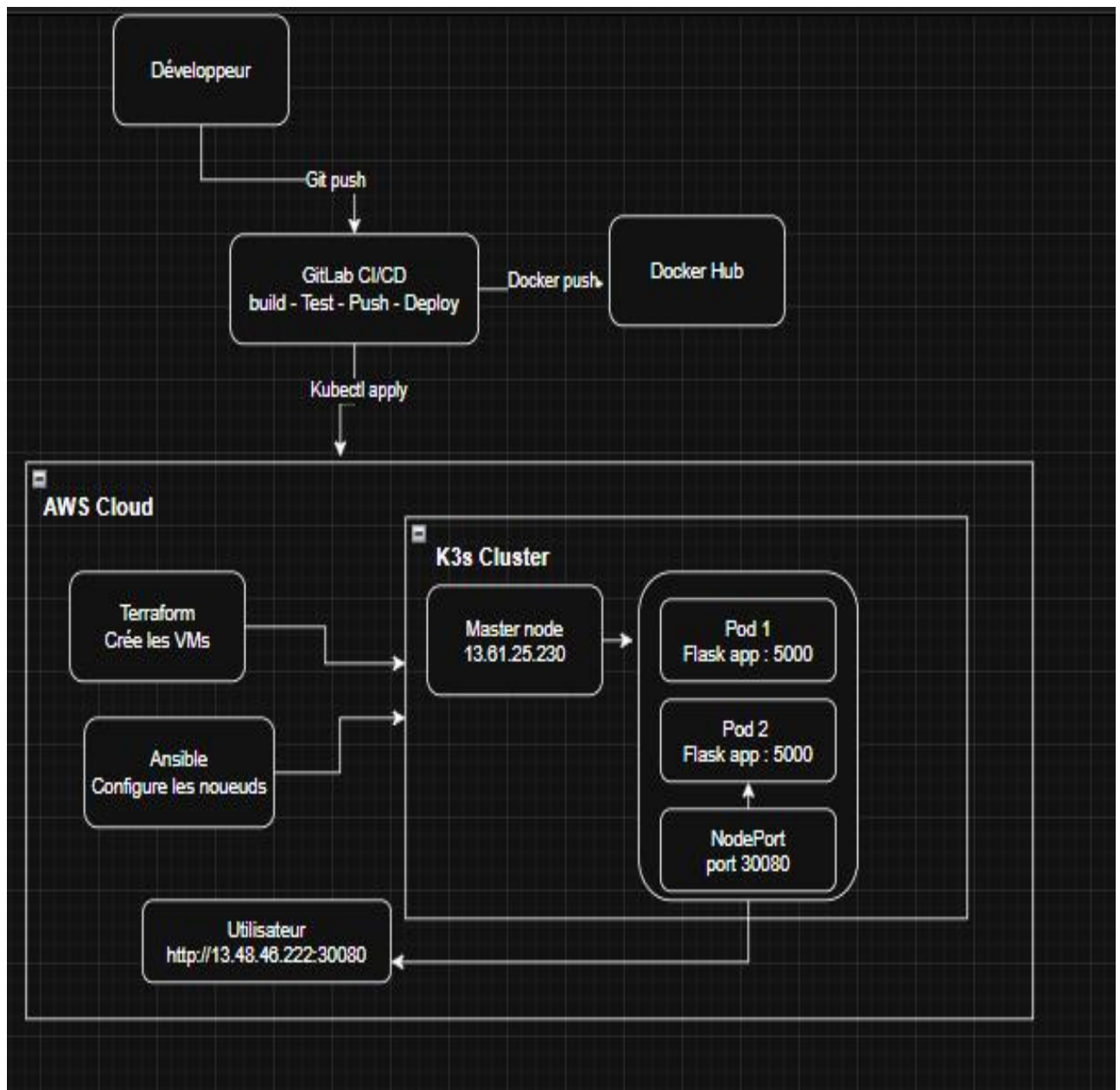


Image 1 : Architecture globale de l'infrastructure DevOps

3.2 Description des composantes

◆ Flux de déploiement

Le développeur effectue un git push vers GitLab. Le pipeline CI/CD se déclenche automatiquement et exécute les 4 stages : build, test, push vers Docker Hub, et déploiement sur le cluster K3s.

◆ Infrastructure cloud

Deux machines virtuelles AWS EC2 (t3.micro) sont créées par Terraform dans la région eu-north-1 (Stockholm). Le master héberge le plan de contrôle K3s et le worker exécute les pods de l'application.

◆ Cluster Kubernetes K3s

K3s est une distribution légère de Kubernetes adaptée aux machines avec peu de ressources. Le master gère le cluster et le worker exécute 2 replicas de l'application Flask, garantissant la haute disponibilité.

4. Implémentation

4.1 Création de l'application

L'application GestiAbsence est développée avec Flask en Python. Elle permet la gestion complète des absences étudiantes via une interface web.

Structure du projet :

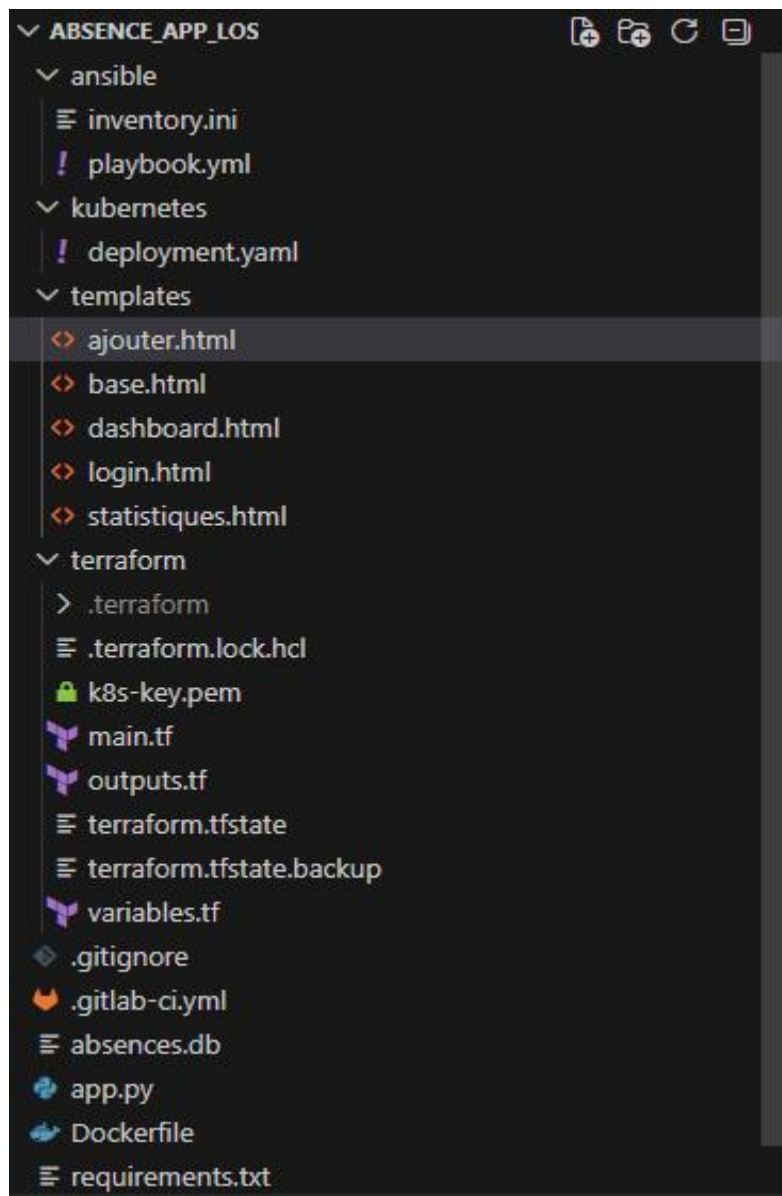


Image 2 : Structure du projet GestiAbsence

L'application offre les fonctionnalités suivantes :

- Authentification administrateur sécurisée
- Tableau de bord avec statistiques globales
- Ajout, modification et suppression d'absences
- Filtrage par étudiant, classe, type et date
- Export des données en format Excel
- Visualisation graphique des absences par classe

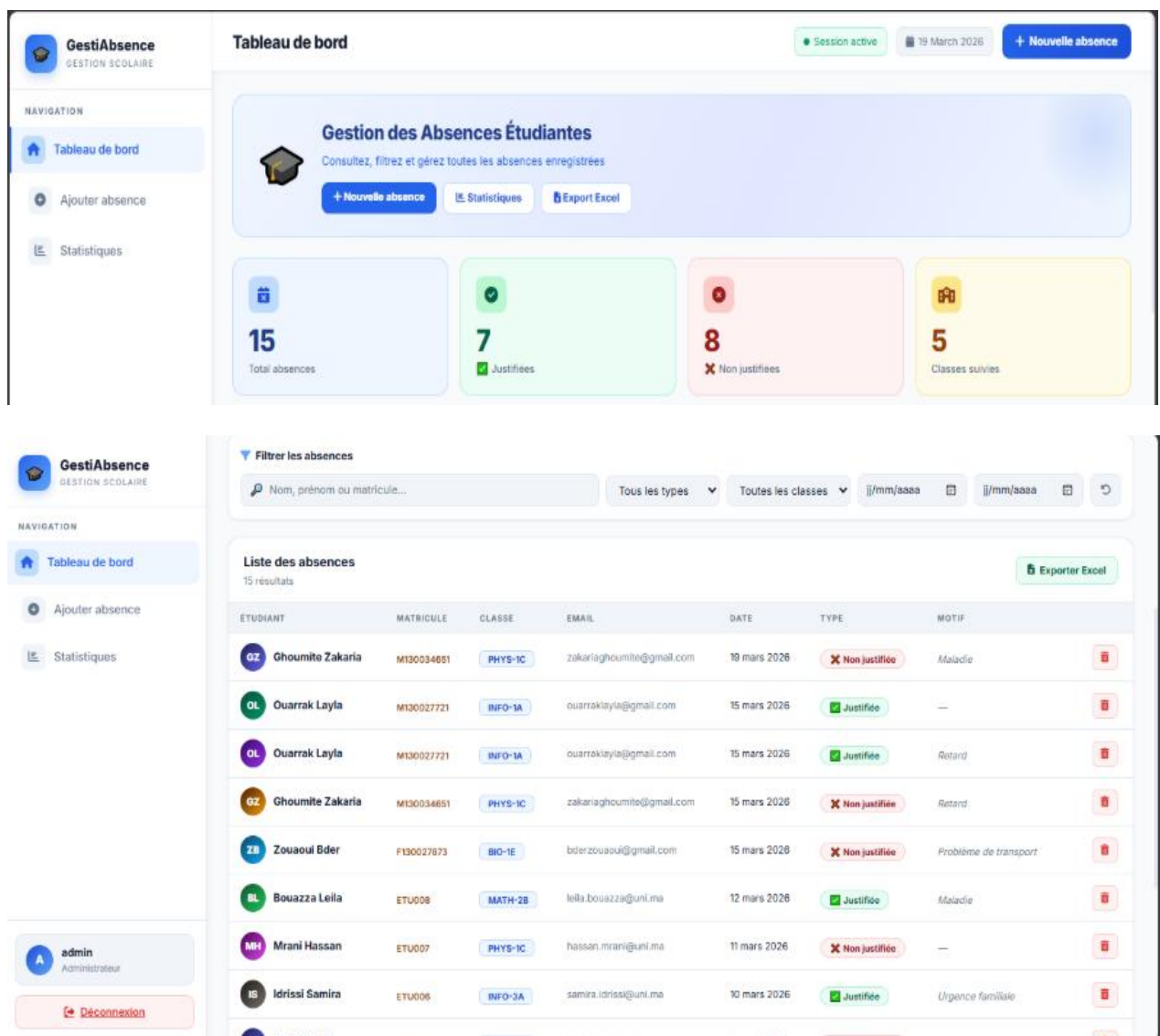


Image 3 : Interface principale de l'application GestiAbsence

GestiAbsence

GESTION SCOLAIRE

NAVIGATION

Tableau de bord

Ajouter absence

Statistiques

admin

Administrateur

Déconnexion

INFORMATIONS ÉTUDIANT

NOM *

Ex: Ouarrak

PRÉNOM *

Ex: Layla

MATRICULE *

M130026001

CLASSE *

— Sélectionner —

EMAIL ÉTUDIANT

etudiant@universite.ma

DÉTAILS DE L'ABSENCE

DATE *

19/03/2026

TYPE D'ABSENCE *

Justifiée

Non justifiée

MOTIF PRÉDÉFINI

Sélectionner un motif...

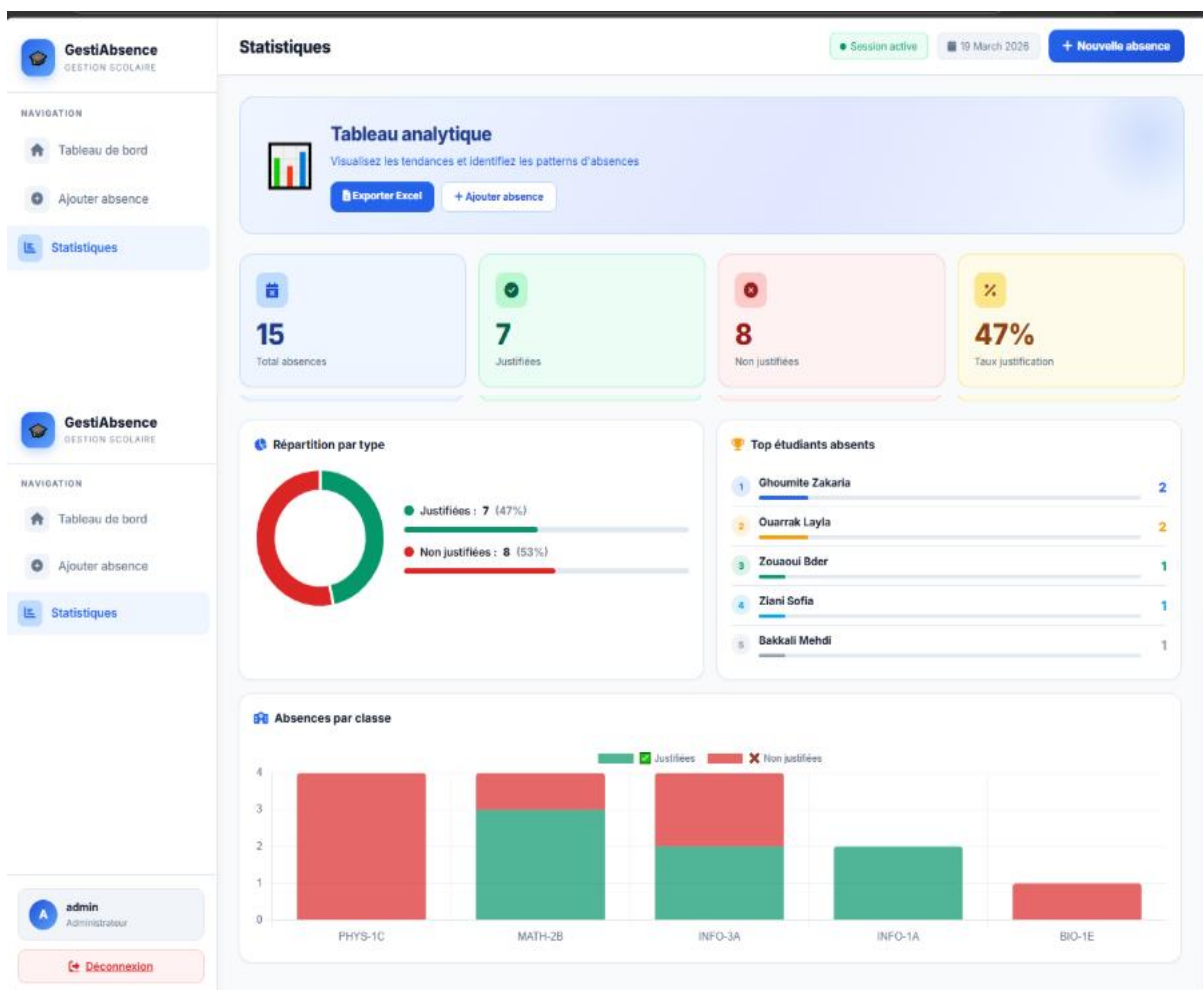
MOTIF PERSONNALISÉ

Ou saisir librement...

Annuler

Enregistrer l'absence

Image 4 : Interface d'ajouter une absence



Statistiques

Session active

19 March 2026

Nouvelle absence

Tableau analytique

Visualisez les tendances et identifiez les patterns d'absences

Exporter Excel

Ajouter absence

15

Total absences

7

Justifiées

8

Non justifiées

47%

Taux justification

Répartition par type

Justifiées : 7 (47%)

Non justifiées : 8 (53%)

Top étudiants absents

Ghoumitte Zakaria

2

Ouarrak Layla

2

Zouaoui Bder

1

Ziani Sofia

1

Bakkali Mehdi

1

Absences par classe

Justifiées

Non justifiées

PHYS-1C

MATH-2B

INFO-3A

INFO-1A

BIO-1E

Image 5 : Page de statistiques analytiques

4.2 Conteneurisation avec Docker

Pour rendre l'application portable et déployable sur n'importe quelle machine, elle a été conteneurisée avec Docker.

Dockerfile :

```
Dockerfile > ...
1  FROM python:3.10
2
3  WORKDIR /app
4
5  COPY . .
6
7  RUN pip install -r requirements.txt
8
9  EXPOSE 5000
10
11 CMD ["python", "app.py"]
```

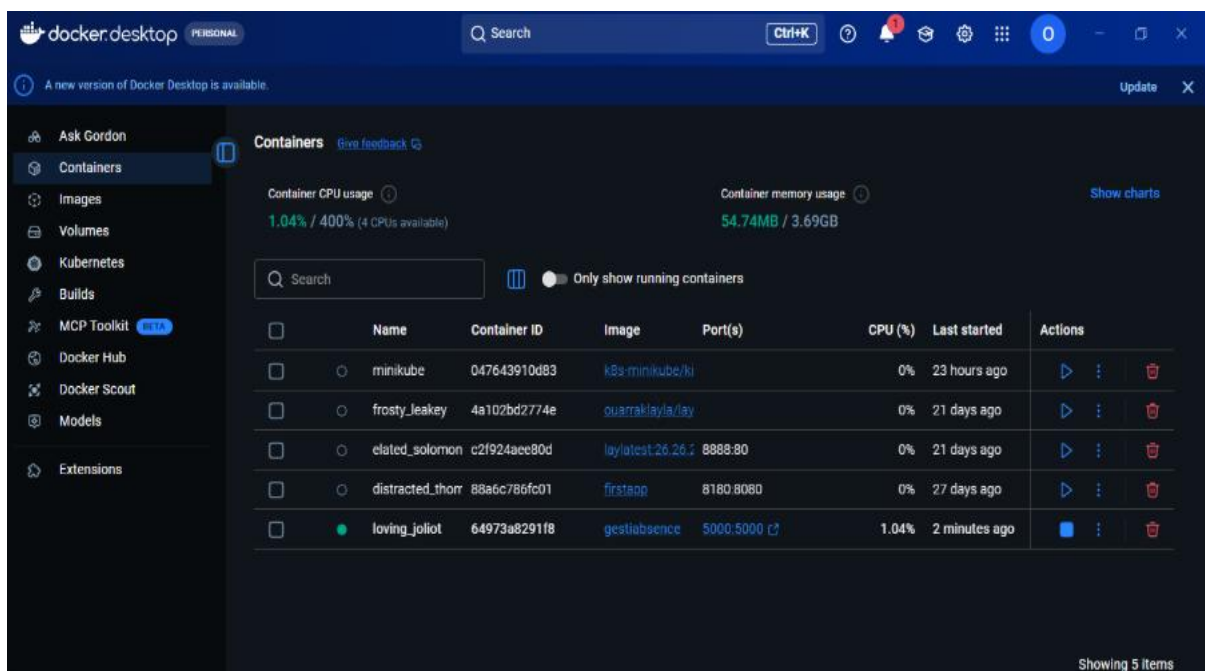
Image 6 : Dockerfile de l'application

Les commandes utilisées pour construire et tester l'image localement :

- `docker build -t gestiabsence .`
- `docker run -p 5000:5000 gestiabsence`

Puis l'image a été publiée sur Docker Hub :

- `docker tag gestiabsence ouarraklayla/gestiabsence:latest`
- `docker login`
- `docker push ouarraklayla/gestiabsence:latest`



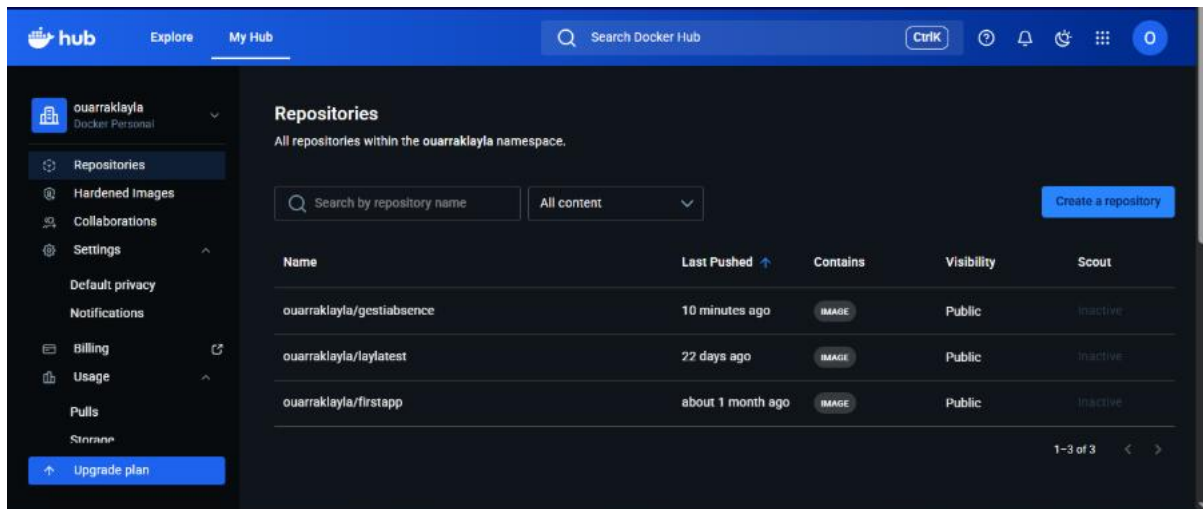


Image 7 : Construction et push de l'image Docker

4.3 Infrastructure as Code avec Terraform

L'infrastructure cloud a été automatisée avec Terraform. Le fichier main.tf définit toutes les ressources AWS nécessaires.

Ressources créées

- 1 Security Group avec les ports 22 (SSH), 6443 (K3s) et 30080 (application)
- 2 instances EC2 t3.micro (master et worker) avec Ubuntu 24.04
- 1 paire de clés SSH (k8s-key.pem)

```

terraform > main.tf
1  terraform {
2    required_providers {
3      aws = {
4        source = "hashicorp/aws"
5        version = "~> 5.0"
6      }
7    }
8  }
9
10 provider "aws" {
11   region = "eu-north-1"
12 }
13
14 resource "aws_security_group" "k8s_sg" {
15   name        = "k8s-security-group"
16   description = "Ports pour Kubernetes"
17
18   ingress {
19     from_port = 22
20     to_port   = 22
21     protocol  = "tcp"
22     cidr_blocks = ["0.0.0.0/0"]
23   }
24   ingress {
25     from_port = 6443
26     to_port   = 6443
27     protocol  = "tcp"
28     cidr_blocks = ["0.0.0.0/0"]
29   }
30   ingress {
31     from_port = 30080
32     to_port   = 30080
33   }
34 }

```

```

    ingress {
      from_port = 30080
      to_port   = 30080
      protocol  = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
    egress {
      from_port = 0
      to_port   = 0
      protocol  = "-1"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}

resource "aws_instance" "master" {
  ami           = "ami-075449515af5df0d1"
  instance_type = "t3.micro"
  key_name      = "k8s-key"
  vpc_security_group_ids = [aws_security_group.k8s_sg.id]
  tags = { Name = "k8s-master" }
}

resource "aws_instance" "worker" {
  ami           = "ami-075449515af5df0d1"
  instance_type = "t3.micro"
  key_name      = "k8s-key"
  vpc_security_group_ids = [aws_security_group.k8s_sg.id]
  tags = { Name = "k8s-worker" }
}

```

```

terraform > variables.tf
1  variable "region" {
2    |   default = "us-east-1"
3    | }

```

```

terraform > outputs.tf
1  output "master_ip" {
2    |   value = aws_instance.master.public_ip
3    | }
4  output "worker_ip" {
5    |   value = aws_instance.worker.public_ip
6    | }

```

Image 8 : Fichiers Terraform (main.tf, variables.tf, outputs.tf)

Commandes exécutées :

- terraform init # Initialisation des providers
- terraform plan # Prévisualisation des ressources
- terraform apply # Création des VMs AWS

```

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_instance.worker: Creating...
aws_instance.master: Creating...
aws_instance.worker: Still creating... [00m10s elapsed]
aws_instance.master: Still creating... [00m10s elapsed]
aws_instance.worker: Creation complete after 14s [id=i-02e4fa029067ae573]
aws_instance.master: Creation complete after 14s [id=i-0a9a87bf6555822ef]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

master_ip = "13.61.25.230"
worker_ip = "13.48.46.222"

C:\Users\Layla Ouarrak\Desktop\Cloud_DeVops\Absence_App_LOS\terraform>

```

Image 9 : Résultat de terraform apply — 2 VMs créées

Résultat : les IPs publiques des machines sont affichées automatiquement :

- master_ip = "13.61.25.230"
- worker_ip = "13.48.46.222"

4.4 Configuration automatique avec Ansible

Après la création des machines, leur configuration a été automatisée avec Ansible via un playbook qui installe Docker, K3s et configure le swap sur les deux nœuds.

Fichier inventory.ini

```
ansible > inventory.ini
1  [master]
2  13.61.25.230 ansible_user=ubuntu ansible_ssh_private_key_file=~/.k8s-key.pem
3
4  [worker]
5  13.48.46.222 ansible_user=ubuntu ansible_ssh_private_key_file=~/.k8s-key.pem
6
7  [all:vars]
8  ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

```
ansible > ! playbook.yml
1  ---
2  - name: Configurer les noeuds K3s
3    hosts: all
4    become: yes
5    tasks:
6
7      - name: Mettre à jour le système
8        apt:
9          update_cache: yes
10
11     - name: Installer Docker
12       apt:
13         name: docker.io
14         state: present
15
16     - name: Démarrer Docker
17       service:
18         name: docker
19         state: started
20         enabled: yes
21
22     - name: Créer le swapfile
23       command: fallocation -l 1G /swapfile
24       args:
25         creates: /swapfile
26
27     - name: Permissions swapfile
28       file:
29         path: /swapfile
30         mode: '0600'
31
32     - name: Formater le swap
```

```

31
32     - name: Formater le swap
33       command: mkswap /swapfile
34       args:
35         creates: /swapfile
36
37     - name: Activer le swap
38       command: swapon /swapfile
39       ignore_errors: yes
40
41 - name: Installer K3s sur le master
42   hosts: master
43   become: yes
44   tasks:
45
46     - name: Installer K3s server
47       shell: curl -sL https://get.k3s.io | sh -
48       args:
49         creates: /usr/local/bin/k3s
50
51     - name: Récupérer le token K3s
52       command: cat /var/lib/rancher/k3s/server/node-token
53       register: k3s_token
54
55     - name: Afficher le token
56       debug:
57         msg: "{{ k3s_token.stdout }}"
58
59 - name: Installer K3s sur le worker
60
61 - name: Installer K3s sur le worker
62   hosts: worker
63   become: yes
64   tasks:
65
66     - name: Installer K3s agent
67       shell: |
68         curl -sL https://get.k3s.io | K3S_URL=https://13.61.25.230:6443 \
69           K3S_TOKEN={{ hostvars['13.61.25.230']['k3s_token']['stdout'] }} sh -
70       args:
71         creates: /usr/local/bin/k3s

```

Image 10 : Fichiers Ansible (inventory.ini , playbook.yml)

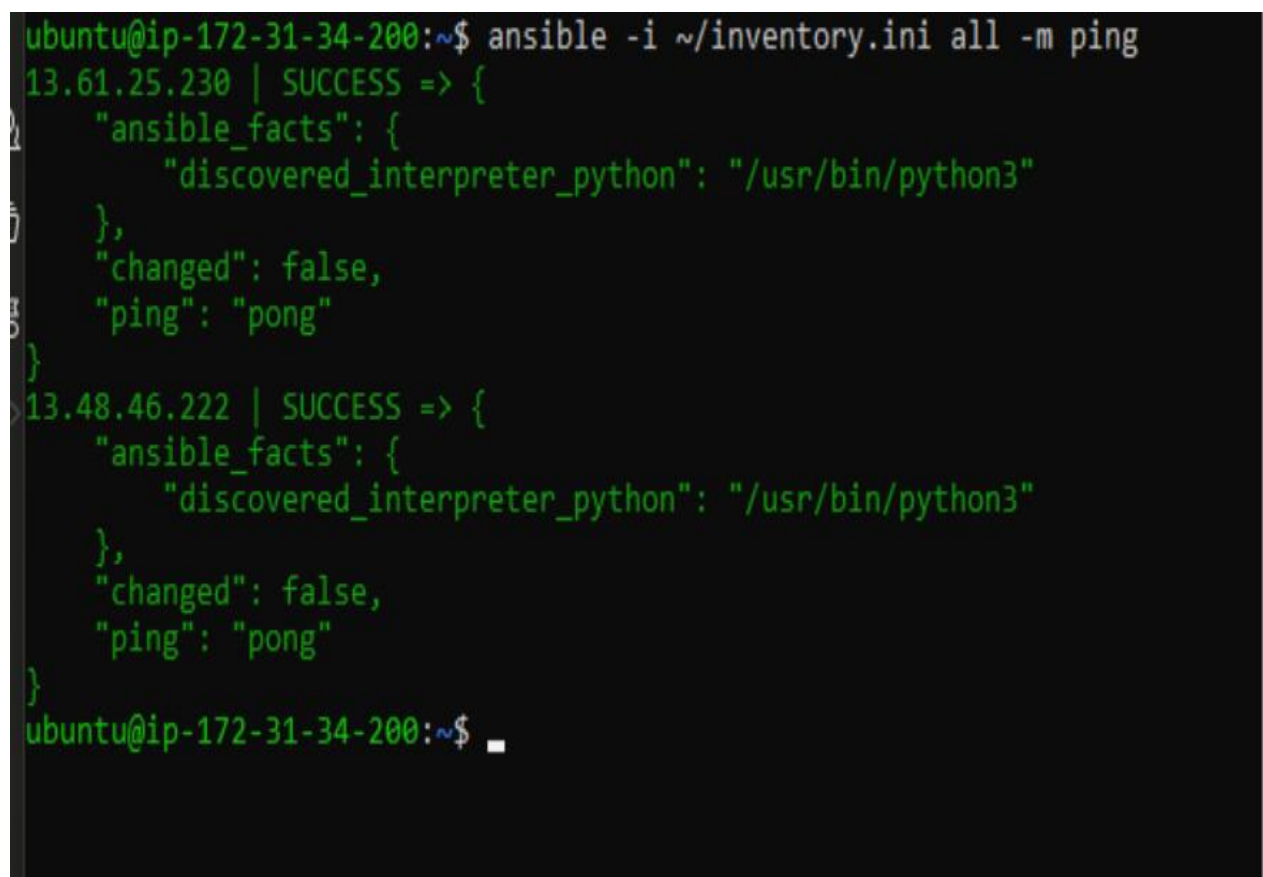
Tâches du playbook

Le playbook automatise les tâches suivantes sur toutes les machines :

- Mise à jour des paquets système
- Installation et démarrage de Docker
- Création du swapfile de 1GB (mémoire de secours)
- Installation de K3s server sur le master
- Installation de K3s agent sur le worker avec le token du master

Commande d'exécution :

- `ansible -i inventory.ini all -m ping` # Test de connectivité
- `ansible-playbook -i inventory.ini playbook.yml`



```
ubuntu@ip-172-31-34-200:~$ ansible -i ~/inventory.ini all -m ping
13.61.25.230 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
13.48.46.222 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu@ip-172-31-34-200:~$
```

Image 11 : Résultat ansible ping — SUCCESS sur les 2 nœuds

4.5 Déploiement avec Kubernetes (K3s)

Le déploiement de l'application sur le cluster K3s utilise un fichier YAML qui définit deux ressources : un Deployment et un Service.

Fichier deployment.yaml :

```
kubernetes > ! deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: gestiabsence
5    labels:
6      app: gestiabsence
7  spec:
8    replicas: 2
9    selector:
10     matchLabels:
11       app: gestiabsence
12    template:
13     metadata:
14       labels:
15         app: gestiabsence
16     spec:
17       containers:
18       - name: gestiabsence
19         image: ouarraklayla/gestiabsence:latest
20         ports:
21         - containerPort: 5000
22
23  ---
24  apiVersion: v1
25  kind: Service
26  metadata:
27    name: gestiabsence-svc
28  spec:
29    type: NodePort
30    selector:
31      app: gestiabsence
32    ports:
33    - port: 5000
34      targetPort: 5000
35      nodePort: 30080
```

Image 12 : Fichier deployment.yaml Kubernetes

```
ubuntu@ip-172-31-34-200:~$ sudo systemctl restart k3s
ubuntu@ip-172-31-34-200:~$ sudo k3s kubectl get nodes --kubeconfig /etc/rancher/k3s/k3s.yaml
NAME                STATUS    ROLES    AGE    VERSION
ip-172-31-34-200    Ready    control-plane   7h46m   v1.34.5+k3s1
ip-172-31-39-248    Ready    <none>         3s      v1.34.5+k3s1
ubuntu@ip-172-31-34-200:~$
```

Image 13 : État des pods et des nœuds K3s

4.6 Pipeline CI/CD avec GitLab

Le pipeline CI/CD est configuré via le fichier `.gitlab-ci.yml`. Il se déclenche automatiquement à chaque git push sur la branche master et exécute 4 stages séquentiels.

Fichier `.gitlab-ci.yml` :

```
.gitlab-ci.yml
1  stages:
2    - build
3    - test
4    - push
5    - deploy
6
7  variables:
8    IMAGE: $DOCKER_USER/gestiabsence:latest
9
10 # — STAGE 1 : Build —
11 build:
12   stage: build
13   image: docker:24
14   services:
15     - docker:24-dind
16   script:
17     - docker build -t $IMAGE .
18   only:
19     - master
20
21 # — STAGE 2 : Test —
22 test:
23   stage: test
24   image: python:3.11-slim
25   script:
26     - pip install flask openpyxl --quiet
27     - python -c "from app import app, init_db; init_db(); print('Tests OK')"
28   only:
29     - master
30
31 # — STAGE 3 : Push Docker Hub —
32 push:
33   stage: push
34   image: docker:24
35   services:
36     - docker:24-dind
37   script:
38     - docker build -t $IMAGE .
39     - echo $DOCKER_PASS | docker login -u $DOCKER_USER --password-stdin
40     - docker push $IMAGE
41   only:
42     - master
43
44 # — STAGE 4 : Deploy sur K3s —
45 deploy:
46   stage: deploy
47   image: alpine/k8s:1.28.3
48   script:
49     - mkdir -p ~/.kube
50     - echo "$KUBECONFIG_CONTENT" | base64 -d > ~/.kube/config
51     - sed -i 's/127.0.0.1/13.61.25.230/g' ~/.kube/config
52     - kubectl config set-cluster default --insecure-skip-tls-verify=true
53     - kubectl apply -f kubernetes/deployment.yaml
54     - kubectl rollout restart deployment/gestiabsence
55     - kubectl get pods
56   only:
57     - master
```

Image 14 : Fichier `.gitlab-ci.yml` complet

Variables CI/CD configurées dans GitLab (Settings → CI/CD → Variables) :



Variables CI/CD </> 4				Révéler les valeurs		Ajouter une variable
Clé ↑	Valeur	Environnements		Actions		
DOCKER_PASS		Tous (par défaut)				
DOCKER_USER		Tous (par défaut)				
K8S_SERVER		Tous (par défaut)				
KUBECONFIG_CONTENT		Tous (par défaut)				

Image 15 : Variables CI/CD dans GitLab

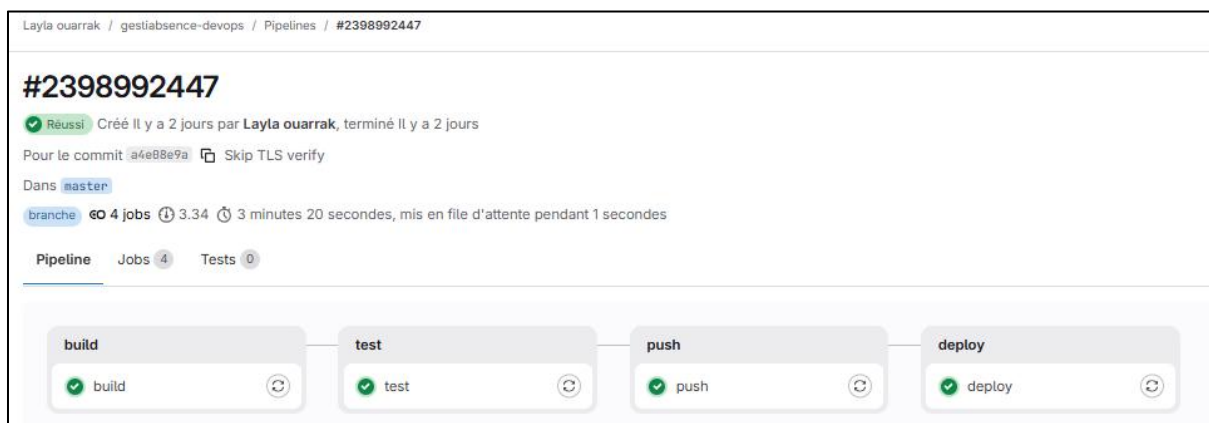


Image 16 : Pipeline GitLab réussi — 4 stages verts

5. Résultats et validation

5.1 Tests réalisés

Plusieurs tests ont été effectués pour valider le bon fonctionnement de l'infrastructure :

Test	Commande / Méthode	Résultat
Connectivité Ansible	ansible -i inventory.ini all -m ping	SUCCESS — pong sur les 2 nœuds
État du cluster K3s	kubectl get nodes	Ready — master et worker
État des pods	kubectl get pods	2/2 Running
Accès à l'application	curl http://13.48.46.222:30080	Redirection vers /login
Pipeline CI/CD	git push origin master	4/4 stages réussis
Navigateur	http://13.48.46.222:30080	Application accessible

Table 3 : Tableau des tests réalisés

5.2 Résultats obtenus

L'ensemble des objectifs du projet ont été atteints avec succès :

- Application Flask déployée et accessible depuis internet
- 2 pods en état Running sur le worker node
- Pipeline CI/CD fonctionnel : chaque git push déclenche un déploiement automatique
- Cluster K3s avec master et worker en état Ready
- Infrastructure créée entièrement par code (Terraform + Ansible)

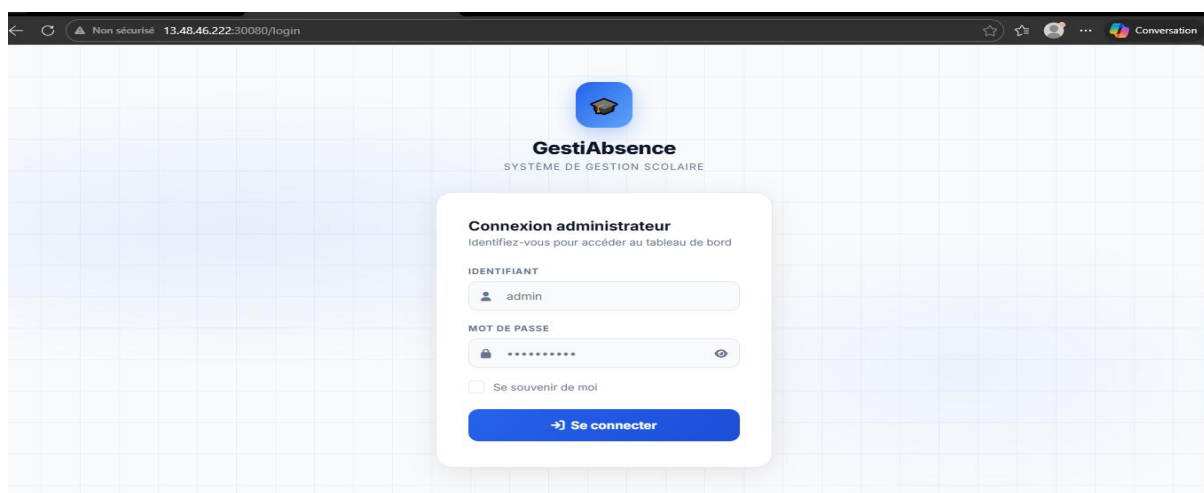


Image 17 : Application accessible sur <http://13.48.46.222:30080>

6. Conclusion et perspectives

6.1 Bilan

Ce projet a permis de mettre en place une infrastructure DevOps complète et fonctionnelle. L'ensemble des objectifs fixés ont été atteints : développement de l'application, conteneurisation avec Docker, provisioning de l'infrastructure avec Terraform, configuration automatique avec Ansible, orchestration avec K3s et automatisation du déploiement avec GitLab CI/CD.

L'architecture mise en place simule fidèlement un environnement de production d'entreprise, avec une séparation claire des responsabilités entre les différents outils.

6.2 Limites

- Machines virtuelles de petite taille (t3.micro - 1GB RAM) limitant les ressources disponibles
- Pas de monitoring mis en place (Prometheus/Grafana) par manque d'espace disque
- Base de données SQLite non persistante entre les redémarrages de pods
- Un seul worker node — pas de véritable haute disponibilité

6.3 Améliorations futures

- Ajouter un système de monitoring avec Prometheus et Grafana
- Utiliser une base de données persistante (PostgreSQL avec PersistentVolume)
- Ajouter des machines plus puissantes pour un environnement de production réel
- Mettre en place des tests unitaires et d'intégration plus complets
- Configurer un domaine personnalisé avec HTTPS (Let's Encrypt)
- Implémenter un inventaire Ansible dynamique pour la scalabilité